Name: Ishaan Vijaykumar Dhore

PRN: 126224310

Roll no.: 29

Panel: G



School of Computer Science and Engineering
Department of Computer Engineering and Technology

T.Y. B. Tech CSE
AY 2026-27 Semester 5

Computer Networks Lab (CSE20110)
Assignment No.: 5

Title: Write a program for TCP Socket Programming: (Language: C/C++/Python/Java).

Aim: Write a C program for wired network using TCP socket to demonstrate

a. Chat application (50% students)
b. Mathematical operations. (Remaining 50% students)

Objectives:

1. To understand the concept of socket programming.

Theory:

1. What is a Socket?

A socket is a software endpoint that enables two-way communication between a client and a server over a network. It is identified by a combination of an IP address and a port number, and it acts as the interface through which an application sends and receives data using an underlying transport protocol such as TCP or UDP. A socket can be thought of as one end of a pipe connecting two programs — once a connection is established, data can flow through this pipe in either direction, much like how a telephone handset allows two people to talk to each other once a call is connected.

2. TCP Socket Primitives at Server and Client

TCP socket programming follows a well-defined sequence of system calls (primitives) on both the server and client side to establish a connection, exchange data, and terminate the session.

On the server side, the primitives are used in the following order:

- socket() — creates a new TCP socket and returns a socket descriptor.
- bind() — associates the socket with a specific IP address and port number on the local machine.
- listen() — puts the socket into a passive, listening state, ready to accept incoming connection requests.
- accept() — blocks until a client connection request arrives, then creates a new socket dedicated to that client.
- send() / recv() — used to exchange data with the connected client.

- close() — terminates the socket and releases the associated resources.

On the client side, the primitives are used in the following order:

- socket() — creates a new TCP socket on the client machine.
- connect() — actively establishes a connection to the server's IP address and port number.
- send() / recv() — used to exchange data with the connected server.
- close() — terminates the socket once communication is complete.

Together, these primitives implement the three-way handshake and data exchange that make TCP a reliable, connection-oriented protocol — the server passively waits for a connection (listen/accept) while the client actively initiates one (connect), after which both sides can freely send and receive data until the connection is closed.

Pseudocode:

Server Program:

2. START
3. Create TCP socket
4. Bind socket to IP address and port number
5. Listen for client connection
6. Accept client connection
7. Receive message from client
8. Display received message
9. Send reply to client
10. Close client connection
11. Close server socket
12. STOP

Client Program:

13. START
14. Create TCP socket
15. Connect to server IP address and port number
16. Input message from user
17. Send message to server
18. Receive reply from server
19. Display server reply
20. Close client socket
21. STOP

Program Code:

Server:

```
#include <iostream>
#include <string>
#include <winsock2.h>

int main() {
    // 0. Start Windows Sockets (Required on Windows)
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);

    // 1. Create socket
    SOCKET server_fd = socket(AF_INET, SOCK_STREAM, 0);

    // 2. Bind to port 8080
    sockaddr_in addr = {AF_INET, htons(8080)};
    addr.sin_addr.s_addr = INADDR_ANY;
    bind(server_fd, (sockaddr*)&addr, sizeof(addr));

    // 3. Listen & Accept
    listen(server_fd, 1);
    std::cout << "Waiting for client...\n";
    SOCKET client_fd = accept(server_fd, nullptr, nullptr);
    std::cout << "Client connected! (Type 'exit' to quit)\n\n";

    // 4. Chat loop
    std::string msg;
    while (true) {
        char buffer[1024] = {0};
        recv(client_fd, buffer, 1024, 0);
        std::cout << "Client: " << buffer << "\n";
```

```
        if (std::string(buffer) == "exit") break;

        std::cout << "You (Server): ";
        std::getline(std::cin, msg);
        send(client_fd, msg.c_str(), msg.length(), 0);
        if (msg == "exit") break;
    }

    // 5. Close & Cleanup
    closesocket(client_fd);
    closesocket(server_fd);
    WSACleanup();
}
```

Client:

```
#include <iostream>
#include <string>
#include <winsock2.h>

int main() {
    // 0. Start Windows Sockets (Required on Windows)
    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);

    // 1. Create socket
    SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);

    // 2. Connect to localhost (127.0.0.1) on port 8080
    sockaddr_in addr = {AF_INET, htons(8080)};
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(sock, (sockaddr*)&addr, sizeof(addr));
```

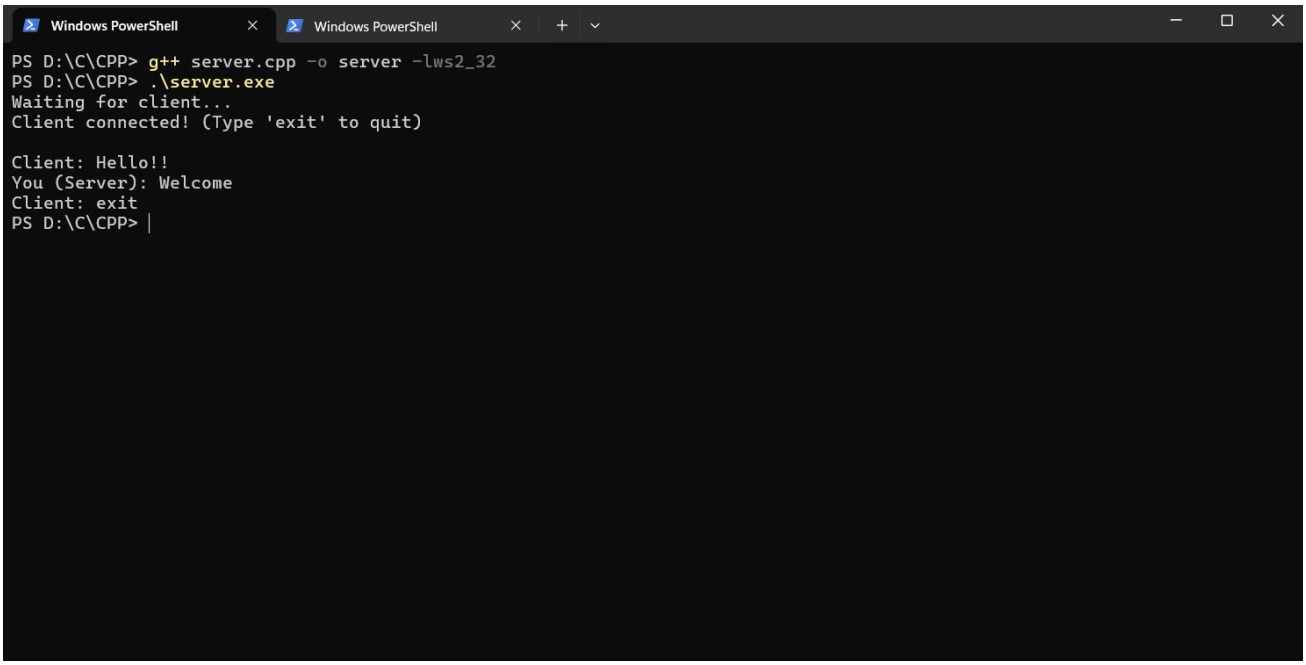
```
    std::cout << "Connected to Server! (Type 'exit' to quit)\n\n";

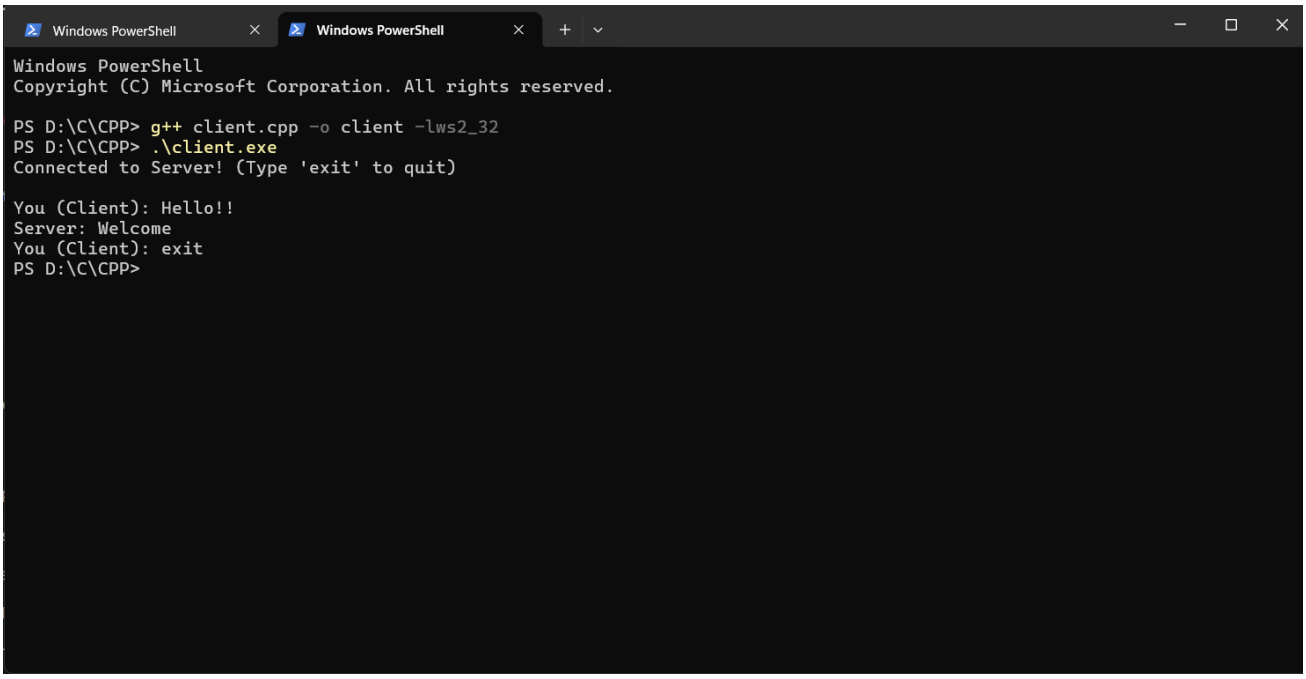
    // 3. Chat loop
    std::string msg;
    while (true) {
        std::cout << "You (Client): ";
        std::getline(std::cin, msg);
        send(sock, msg.c_str(), msg.length(), 0);
        if (msg == "exit") break;

        char buffer[1024] = {0};
        recv(sock, buffer, 1024, 0);
        std::cout << "Server: " << buffer << "\n";
        if (std::string(buffer) == "exit") break;
    }

    // 4. Close & Cleanup
    closesocket(sock);
    WSACleanup();
}
```

Output Screenshots:





Students Observation:

Running this lab helped me see how a TCP connection actually comes together in practice, not just on paper. After starting the server, it sat listening on port 8080 until the client connected, confirming the three-way handshake worked as expected. The message sent from the client showed up correctly on the server, and the reply came back without any issues — no data loss or reordering, which fits with TCP being a reliable, connection-oriented protocol. I also noticed that the server has to be up and listening before the client tries to connect, otherwise the connection just fails. Overall, the exercise made the socket() → bind() → listen() → accept() → connect() flow feel a lot more concrete than just reading about it.

FAQ:

1) What is a TCP Socket?

A TCP socket is a communication endpoint that uses the Transmission Control Protocol (TCP) to provide reliable, connection-oriented, and ordered delivery of data between two devices on a network. It is created using

the SOCK_STREAM socket type, and before any data can be exchanged, a connection must first be established between the client and server through the TCP three-way handshake. TCP sockets guarantee that data arrives in the order it was sent and retransmit any lost packets automatically, which makes them suitable for applications where accuracy and reliability matter more than raw speed.

2) What are the main steps involved in TCP Socket Programming?

TCP socket programming on the server side involves creating a socket, binding it to an IP address and port, listening for incoming connections, accepting a client connection, and then exchanging data using send()/recv() before finally closing the socket. On the client side, it involves creating a socket, connecting it to the server's IP address and port, exchanging data with the server, and closing the socket once communication is complete. This sequence of socket(), bind(), listen(), accept() on the server, and socket(), connect() on the client, forms the standard skeleton of any TCP client-server application.

3) What is the role of IP address and port number in TCP Socket Programming?

The IP address identifies the specific device (host) on the network that a socket belongs to, while the port number identifies the specific application or process running on that device that should receive the data. Together, the IP address and port number form a unique socket address (for example, 192.168.1.10:8080) that allows the operating system to correctly deliver incoming data to the right application, even when multiple applications are running on the same machine. The server binds to a known IP address and port so that clients know exactly where to connect, while the client's own IP and port are typically assigned automatically by the operating system.

4) Why is TCP called a connection-oriented protocol?

TCP is called a connection-oriented protocol because a dedicated end-to-end connection must be explicitly established between the sender and receiver before any data is exchanged, and this connection is also explicitly terminated once communication is complete. This is done through the three-way handshake (SYN, SYN-ACK, ACK), which synchronizes both sides and confirms that a reliable channel exists before data transfer begins. Because the connection state is maintained throughout the session, TCP can guarantee ordered delivery, detect lost packets, and retransmit them — something that is not possible in a connectionless protocol where each packet is sent independently with no guarantee of delivery.

5) What is the difference between TCP and UDP socket primitives?

Although both TCP and UDP sockets are created using the same socket() call, the sequence of primitives used afterward differs significantly because TCP is connection-oriented while UDP is connectionless. The table below summarizes the key differences.

Parameter	**TCP Socket Primitives**	**UDP Socket Primitives**
Connection	Connection-oriented: requires connect() (client) and listen()/accept() (server) before data transfer.	Connectionless: no connect()/accept() required; sendto()/recvfrom() used directly.
Reliability	Reliable — guarantees delivery, ordering, and error-checking via acknowledgements and retransmission.	Unreliable — no guarantee of delivery, ordering, or duplicate protection.
Key primitives	socket(), bind(), listen(), accept(), connect(), send(), recv(), close()	socket(), bind(), sendto(), recvfrom(), close()
Overhead	Higher, due to connection setup, acknowledgements and flow control.	Lower, since there is no connection setup or acknowledgement mechanism.
Typical use	File transfer, web browsing, email — where accuracy matters.	Video/audio streaming, DNS queries — where speed matters more than accuracy.